Rhyme: Evaluating LLM-Based Incident Correlation

Author: Jeff Campbell, with assistance from Claude Opus 4.6

Abstract

AI-assisted incident response tools increasingly surface "similar past incidents" to help on-call engineers. These tools risk producing red herrings — incidents that share symptoms but have different proximal causes, leading engineers to pursue wrong remediations. In such cases, the incidents "rhyme" (share symptoms) without "repeating" (sharing a cause). We therefore present Rhyme, a benchmark inspired by the phrase frequently attributed to Mark Twain ("History doesn't repeat itself, but it often rhymes"). Rhyme evaluates how well language models distinguish cause-similar from symptom-similar incidents across 20 cause classes in cloud-native environments. We evaluate 9 models from OpenAI, Anthropic, Google, and DeepSeek against BM25 and TF-IDF baselines across three tasks: full-corpus retrieval, reasoning-only re-ranking, and remediation selection. Our key finding is that LLMs do not meaningfully outperform BM25 keyword matching on incident retrieval (all models score 0.78–0.82 precision@10 vs BM25's 0.81), but dramatically outperform it on remediation selection (72–96% correct vs 16% random), with significant safety-relevant variation across models. We propose that incident correlation systems should use keyword retrieval for candidate selection and reserve LLM reasoning for re-ranking and remediation — a hybrid architecture that achieves better results at 30x lower cost. We also introduce an adversarial style probe methodology for validating synthetic benchmark corpora and a web-based labeling tool for evaluating models against organization-specific incident data.

---

1. Introduction

The adoption of AI in incident response has accelerated rapidly. Observability platforms, incident management tools, and internal SRE teams are building features that automatically surface similar past incidents when a new incident occurs. The premise is sound: if an engineer can see how a similar problem was resolved before, they can respond faster and more effectively.

However, this premise contains a dangerous assumption — that "similar" means "caused by the same thing." In software architecture, incidents frequently share symptoms while having entirely different proximal causes. A retry storm and a genuine downstream slowdown both manifest as elevated latency and 5xx error rates. A memory leak and traffic-driven allocation pressure both show rising memory usage and OOM kills. A code regression and a configuration regression both produce step-function error rate increases after a change.

We call these **red herrings** — incidents that a correlation tool confidently surfaces as similar, but that would lead an engineer to the wrong diagnosis. Red herrings are not merely unhelpful; they are actively harmful. An engineer who applies the remediation from a symptom-similar but cause-different incident wastes time, and in some cases makes the situation worse (e.g., scaling up a service that is in a retry storm feeds more capacity to the storm).

Despite the proliferation of AI-assisted incident tools, no existing benchmark specifically evaluates the red-herring failure mode. General-purpose retrieval benchmarks (BEIR, MTEB) evaluate semantic similarity without domain-specific notions of cause. AIOps benchmarks focus on anomaly detection, root cause localization within a single incident, or log parsing — not cross-incident correlation. IncidentBench (Microsoft, 2024) evaluates triage and summarization, not matching incidents to each other.

This paper makes three contributions:

1. **Rhyme**, a benchmark with 1,000 synthetic incidents across 20 cause classes, 200 queries stratified by confusability tier, and deterministic scoring that explicitly measures the red-herring failure mode
2. **Empirical results** from 9 models across 4 providers showing that LLMs do not outperform keyword matching on retrieval but dramatically outperform it on remediation, with safety-relevant variation across models
3. **A methodology** for validating synthetic benchmark quality using adversarial style probes and real-incident style transfer

---

2. Background and related work

2.1 Incident correlation in practice

When an incident occurs, the on-call engineer typically follows a pattern: assess symptoms, form a hypothesis about the proximal cause, investigate, and remediate. Surfacing similar past incidents accelerates the hypothesis step — but only if the surfaced incidents are genuinely similar in cause, not just in symptoms.

The distinction between symptom similarity and cause similarity is well-understood in SRE practice but poorly formalized. Two incidents can exhibit identical golden signals (latency, error rate, traffic, saturation) while differing in their proximal cause. The proximal cause determines the correct remediation; the symptoms alone do not.

We deliberately use "proximal cause" rather than "root cause" throughout this work. Root cause analysis implies a single underlying cause, which is often reductive in complex systems. Proximal cause — the most immediate technical failure that the responding team can act on — better matches what SREs need from a correlation tool.

2.2 Existing benchmarks

AIOps benchmarks. RCAEval (Pham et al.) provides 735 fault injection cases across microservice systems with labeled fault types. OpenRCA (Microsoft) offers 335 failure cases with telemetry data. Both focus on root cause localization within a single incident (which component failed?) rather than cross-incident correlation (have we seen this before?).

Retrieval benchmarks. BEIR (Thakur et al., 2021) and MTEB (Muennighoff et al., 2023) evaluate general retrieval quality across domains. They would score a symptom-similar match as correct, making them unsuitable for evaluating cause-aware correlation.

Incident management benchmarks. IncidentBench (Microsoft, 2024) evaluates LLMs on incident triage, assignment, and summarization. It does not evaluate cross-incident matching or the red-herring problem.

Software failure studies. The FAIL database (2024) catalogs 2,457 software failures with an academic dependability taxonomy. Yuan et al. (OSDI '14) analyzed 198 distributed system bugs in detail. Both provide valuable data for taxonomy validation but are not evaluation benchmarks.

2.3 Synthetic benchmark risks

Synthetic benchmarks carry a structural risk: the generator, the taxonomy, and the evaluator may share assumptions, creating a closed loop where the benchmark measures itself rather than real capability. This risk is well-documented in the broader ML evaluation literature and is particularly acute when LLMs are used to generate the test data.

We address this risk through three mechanisms: (1) the fingerprint schema used for ground-truth construction is never visible to models under test, (2) an adversarial style probe validates that cause class cannot be predicted from writing style alone, and (3) the taxonomy is validated against real-world incident data from three independent sources.

---

3. Benchmark design

3.1 Taxonomy

Rhyme defines 20 cause classes across 8 families:

Family	Classes	Count
Dependency	retry_storm, downstream_slowdown, dependency_outage, dns_resolution_failure, certificate_expiry, connection_pool_exhaustion	6
Resource	memory_leak, traffic_allocation, cpu_throttling, disk_pressure	4
Deploy	code_regression, config_regression	2
Network	network_partition, load_balancer_misconfiguration	2
Traffic	traffic_spike	1
Amplification	cascading_timeout	1
Data/Schema	schema_migration_failure, data_corruption	2
Timing	clock_skew, race_condition	2

Each class is paired with one or more **confusable partners** — classes that share observable symptoms but require different remediation. These pairs are assigned difficulty tiers:

- **Hard pairs** share the same symptom profile and require careful reasoning to distinguish (e.g., retry_storm vs. cascading_timeout — both show elevated latency and timeout errors, but one involves request amplification and the other does not)
- **Medium pairs** share partial symptoms but have at least one clear discriminating signal

The taxonomy was validated against 3,087 real cloud provider incidents (AWS, Azure, GCP), 32 classified incidents from public postmortems (danluu/post-mortems), and fault injection patterns from RCAEval. Every cause class has at least one real-world validation source.

3.2 Corpus

The corpus contains 1,000 synthetic incidents (50 per class). Each incident consists of:

- **Model-visible payload:** responder summary (2–5 sentences), 5–15 sampled alerts with timestamps and severity, 20–100 sampled log lines, a small topology fragment, and incident timestamps
- **Hidden labels:** cause class, confusability tier, remediation vocabulary (canonical fix, symptom-masking fix, harmful fix), and a fingerprint vector

Models under test see only the payload. They never see the cause class, fingerprint, or remediation labels.

Generation pipeline. Incidents are generated in three stages:

1. Archetype templates define the structural skeleton for each cause class: fingerprint shape (latency pattern, error distribution, topology, temporal onset, amplification ratio), alert templates, and log templates
2. LLM-generated prose pools (20 summaries, 25 alerts, 35 log lines per class) provide vocabulary and phrasing variation. These were generated using real AWS, Azure, and GCP incident reports as style examples to produce realistic status-page language
3. Post-processing applies cross-class noise injection (20–40% of alerts and logs come from randomly selected other classes) and summary style randomization (5 structural variants)

Adversarial style probe. To validate that the corpus does not contain stylistic patterns that correlate with cause class, we train a logistic regression classifier on content-stripped text (all words replaced with token-type markers W/N/P). The probe gates on summary-only stripped character n-grams: accuracy must not exceed random + 20 percentage points. The current corpus scores 19.1% against a 25% threshold (5% random baseline + 20pp), indicating no detectable stylistic leakage in the summaries.

Full-text probe scores remain elevated (~45%) due to inherent formatting differences in alert and log content across incident types. This is expected — different failure modes produce genuinely different log formats, and models *should* use that information. The concern is when *style* (sentence structure, punctuation patterns) rather than *content* (error messages, metric values) is sufficient to predict cause class.

3.3 Query set

200 queries are selected from the corpus and stratified by confusability tier: 60 easy (30%), 80 medium (40%), 60 hard (30%). An additional 40 private-slice queries (not published) use a different random seed for training-set contamination detection.

Each query has a ground-truth set of correct matches (same cause class) and tempting wrong matches (confusable partner class with high symptom similarity).

3.4 Tasks

Task 1: Full-corpus retrieval. The model receives a query incident and the full corpus (1,000 payloads) and must return a ranked list of the 10 most cause-similar incidents with confidence scores [0, 1]. This tests both retrieval and reasoning capability. Token cost: ~75,000 tokens per query.

Task 2: Reasoning-only retrieval. A BM25 pre-retriever narrows the corpus to 20 candidates. The model re-ranks these 20 into a top-10 with confidence scores. This isolates reasoning quality from retrieval quality by controlling for the candidate set. Token cost: ~2,000 tokens per query.

Task 3: Remediation. Given a query incident and its top-3 retrieved matches, the model selects the best remediation from 5 multiple-choice options: the canonical fix, a symptom-masking fix, a fix that would worsen the situation, and two plausible distractors (canonical fixes from other cause classes). This tests whether the model understands the incident well enough to recommend appropriate action. Token cost: ~500 tokens per query.

3.5 Metrics

All core metrics are deterministic. No LLM-judge is used in scoring.

- **Retrieval@k (per tier):** fraction of queries where at least one correct-cause match appears in the top-k
- **Cause-match precision@k (per tier):** fraction of top-k results sharing the query's cause class, reported as lift over class base rate to normalize for class-frequency imbalance
- **Confusion matrix:** for each query class, which wrong classes dominate the top-k predictions. Diagnostic rather than scalar — shows *where* a model fails, not just *how much*
- **Expected Calibration Error (ECE):** bucketed comparison of confidence scores vs actual precision. Models that claim 90% confidence should be correct ~90% of the time
- **Remediation grade:** distribution over {fixed, no-op, masks-symptom, would-worsen}. The would-worsen rate is safety-critical and reported separately
- **Efficiency:** tokens per query, tokens per correct match, precision per 1,000 tokens. Enables quality-vs-cost comparison across prompting strategies

There is no composite headline score. Scores are reported as per-tier vectors. This is deliberate: composites hide the red-herring failure mode that the benchmark exists to measure.

---

4. Experimental setup

4.1 Models tested

We evaluated 9 models from 4 providers via OpenRouter, plus BM25 and TF-IDF baselines:

Model	Provider	Notes
GPT-4o	OpenAI	Flagship multimodal
GPT-4o mini	OpenAI	Cost-optimized
GPT-4.1 nano	OpenAI	Smallest/fastest
Claude Sonnet 4	Anthropic	Mid-tier
Claude Haiku 3.5	Anthropic	Fast/cheap
Claude Opus 4	Anthropic	Flagship
Gemini 2.0 Flash	Google	Previous-gen fast
Gemini 2.5 Flash	Google	Current-gen fast
DeepSeek V3	DeepSeek	Open-weights
BM25	—	Keyword matching baseline
TF-IDF cosine	—	Vector similarity baseline
Random	—	Floor

All LLM evaluations used Task 2 (reasoning-only, 80 queries) and Task 3 (remediation, 80 queries) against a corpus of 400 incidents (20 per class). Baselines were additionally evaluated on Task 1 against a full 1,000-incident corpus with 200 queries.

4.2 Adapter design

Models were connected via a subprocess protocol: the harness writes JSON to the adapter's stdin, the adapter returns JSON on stdout. Each adapter converts the Rhyme query format into the provider's API format and parses the response. A shared parsing library handles output format variations across models (code fences, thinking tags, prose wrapping, key ordering).

All LLM adapters used zero-shot prompting with the instruction: *"Find the most similar incidents based on PROXIMAL CAUSE similarity (not just symptom overlap). Two incidents share a proximal cause if the same fix would apply to both."*

4.3 Cost

Total evaluation cost across all 9 models on Tasks 2 and 3: approximately $25 via OpenRouter. Task 1 evaluation for LLMs was not performed at scale due to cost (~$50 per model for 200 queries against 1,000 incidents).

---

5. Results

5.1 Retrieval precision (Task 2)

Model	**Precision@10**	**Retrieval@10**	**ECE**
GPT-4.1 nano	0.818	1.000	0.037
GPT-4o	0.811	1.000	0.105
Gemini 2.5 Flash	0.805	1.000	0.102
BM25 (baseline)	**0.805**	**1.000**	**0.127**
DeepSeek V3	0.804	1.000	0.115
GPT-4o mini	0.802	1.000	0.082
Claude Sonnet 4	0.802	1.000	0.091
Gemini 2.0 Flash	0.789	0.975	0.132
Claude Haiku 3.5	0.784	1.000	0.106
Claude Opus 4	0.782	1.000	0.119
TF-IDF cosine	0.554	0.955	0.059
Random	0.037	0.340	0.461

All LLMs cluster in a narrow band (0.782–0.818) centered on the BM25 baseline (0.805). No model achieves a statistically significant advantage over keyword matching on this task.

5.2 Remediation accuracy (Task 3)

Model	Correct fix	No-op	Masks symptom	Would worsen
Claude Opus 4	**96%**	4%	0%	**0%**
GPT-4o	95%	5%	0%	0%
GPT-4o mini	95%	4%	0%	1%
Gemini 2.5 Flash	94%	6%	0%	0%
Claude Sonnet 4	94%	6%	0%	0%
Claude Haiku 3.5	94%	6%	0%	0%
GPT-4.1 nano	90%	10%	0%	0%
Gemini 2.0 Flash	89%	11%	0%	0%
DeepSeek V3	72%	24%	0%	**4%**
Random	16%	44%	20%	9%

Remediation accuracy shows meaningful differentiation across models (72%–96%), unlike retrieval precision. Claude Opus 4 achieves the highest accuracy (96%) with zero harmful suggestions. DeepSeek V3 is notably weaker on remediation (72% correct) and is the only model with a non-trivial harmful suggestion rate (4%).

5.3 Calibration

ECE ranges from 0.037 (GPT-4.1 nano) to 0.132 (Gemini 2.0 Flash) — a 3.5x spread. Well-calibrated confidence scores are important for production use because responders use them to decide how much to trust the suggestion. A model that claims 90% confidence but is correct only 70% of the time will erode trust quickly.

GPT-4.1 nano's calibration (0.037) is notable: it has the most trustworthy confidence scores despite being the smallest and cheapest model tested.

5.4 Baseline comparison (Task 1, full corpus)

On the full 1,000-incident corpus with 200 queries:

Baseline	Precision@10 (easy)	(medium)	(hard)	ECE
BM25	0.815	0.799	0.808	0.127
TF-IDF	0.563	0.588	0.502	0.059
Random	0.038	0.039	0.035	0.461

BM25 shows the expected tier gradient (hard slightly lower than easy), confirming that the confusable pairs create meaningful difficulty variation.

5.5 Token efficiency

Model	Tokens/query	Precision/1k tokens
GPT-4.1 nano	1,875	0.436
GPT-4o mini	1,879	0.427
GPT-4o	1,879	0.432
DeepSeek V3	1,933	0.416
Claude Opus 4	2,163	0.362
Claude Sonnet 4	2,186	0.367
Claude Haiku 3.5	2,186	0.359
Gemini 2.0 Flash	2,243	0.352
Gemini 2.5 Flash	2,307	0.349

OpenAI models are slightly more token-efficient (~1,875 tokens/query) than Anthropic (~2,186) and Google (~2,275) models. Since precision is comparable, this translates to ~15% cost savings for the same quality.

---

6. Discussion

6.1 LLMs don't add retrieval value — but they add reasoning value

The central finding is a decomposition: incident correlation separates into retrieval (finding candidates) and reasoning (evaluating candidates), and LLMs are strong at the second but not the first.

On Task 2, every model scores within ±0.02 of BM25. This means that given BM25's top-20 candidates, LLMs can re-rank them about as well as BM25 ranked them in the first place. The LLMs are not discovering new signal in the incident text — they are processing the same lexical overlap that BM25 uses, with comparable effectiveness but at ~2,000 tokens of cost per query.

On Task 3, the story reverses. LLMs achieve 72–96% accuracy on a 5-option multiple-choice remediation task where random performance is 20%. This requires understanding what the incident *is*, not just what it *looks like*. The ability to distinguish "apply circuit breaker" from "scale up the downstream service" for a retry storm requires domain reasoning that keyword matching cannot perform.

Architectural implication: Incident correlation systems should use a two-stage architecture: (1) BM25 or similar keyword retrieval for fast, cheap candidate selection, (2) LLM reasoning for re-ranking candidates and recommending remediation. This achieves comparable or better quality at 30x lower cost than full-corpus LLM retrieval.

6.2 Model size doesn't predict performance

Claude Opus 4 (the largest and most expensive Anthropic model) scores lower on retrieval (0.782) than Claude Haiku 3.5 (0.784) and GPT-4o mini (0.802). This pattern — larger models performing comparably or worse on retrieval while excelling on remediation — suggests the two tasks require different capabilities.

Retrieval in this benchmark may be more about instruction-following (produce valid JSON with correct incident IDs) than about deep reasoning. Larger models that are more prone to nuanced interpretation may actually perform worse on the mechanical task of ranking by lexical similarity.

Remediation, by contrast, rewards domain understanding. Opus achieves 96% correct (the highest) while nano achieves 90%. The 6-percentage-point gap suggests that model scale does help with reasoning about which fix applies to which situation — but the gap is smaller than the gap between any LLM and random (90% vs 16%).

6.3 Safety-relevant variation

DeepSeek V3 is the only model with a meaningful harmful-suggestion rate (4%). In a production system where remediation suggestions are presented to on-call engineers, a 4% rate of "this fix will make things worse" is significant — over 200 incidents per year, that's 8 harmful suggestions. Combined with its lower correct-fix rate (72% vs 89–96% for other models), DeepSeek V3 would require additional safeguards (confidence thresholds, human review gates) before deployment in a remediation-suggestion role.

This safety differentiation is invisible in retrieval precision scores, where DeepSeek V3 (0.804) looks comparable to GPT-4o (0.811). The remediation task reveals a meaningful quality gap that retrieval metrics miss.

6.4 Calibration matters more than precision

For production deployment, a model's calibration may matter more than its raw precision. An engineer using a correlation tool will see confidence scores alongside suggestions. If those scores are unreliable, the tool's value degrades regardless of its average precision.

GPT-4.1 nano's ECE of 0.037 means its confidence scores are almost perfectly calibrated — when it says 80% confident, it's correct about 80% of the time. Gemini 2.0 Flash's ECE of 0.132 means a 13-percentage-point gap between claimed and actual confidence, which would require recalibration before production use.

6.5 BM25's strength is a feature, not a bug

BM25 at 0.805 precision on synthetic data might suggest the benchmark is too easy. We argue the opposite: BM25's strength reflects a genuine property of incident correlation. Incidents of the same type share vocabulary — memory leaks mention "OOM," "heap," and "GC"; certificate expiries mention "x509," "TLS," and "handshake." This lexical signal is strong and should be exploited.

The benchmark's value is not in differentiating retrieval quality (where all methods converge) but in differentiating reasoning quality (where models diverge on remediation) and measuring safety properties (where DeepSeek's harmful-suggestion rate is a real finding). A benchmark where every model scored 0.99 on retrieval would still be valuable if it revealed that one model suggests harmful remediations 4% of the time.

---

7. Limitations

7.1 Synthetic corpus

The corpus is generated from archetype templates with LLM prose variation. Despite adversarial style probing and real-incident style transfer, synthetic incidents do not capture the full messiness of real incident reports: inconsistent formatting, org-specific jargon, incomplete information, evolving understanding during response, and multiple concurrent incidents.

BM25's high precision (0.805) on synthetic data may overstate its performance on real data, where vocabulary overlap between cause-similar incidents is noisier. Conversely, LLMs' comparable precision may understate their advantage on real data, where their ability to generalize beyond lexical patterns would be more valuable.

7.2 Taxonomy dependence

The 20-class taxonomy is the benchmark's ground truth. A model that disagrees with the taxonomy — that considers two incidents to share a cause when the taxonomy says they don't — is graded as wrong regardless of whether the model is actually right. The taxonomy was validated against real-world data, but it remains a model of reality, not reality itself.

7.3 Single-turn evaluation

All tasks are single-turn. Real incident investigation is iterative: engineers query monitoring systems, read logs, form and revise hypotheses. A model that performs poorly on single-turn retrieval might perform well in an agentic setting where it can ask follow-up questions. This benchmark does not measure that capability.

7.4 Narrow domain

The taxonomy covers cloud-native architecture. Results may not generalize to mainframes, data pipelines, embedded systems, IoT, or non-HTTP architectures.

7.5 Score compression

The narrow precision band (0.782–0.818) means the ranking of models is likely unstable across random seeds. We do not claim that GPT-4.1 nano is definitively better than Claude Sonnet 4 on retrieval. We do claim that no tested model significantly outperforms BM25 on retrieval, and that remediation scores show stable differentiation.

7.6 Finite useful life

Published benchmarks are vulnerable to training-set contamination. As models are trained on more web data, scores on Rhyme will inflate without corresponding capability gains. The private query slice provides some protection, but the benchmark should be versioned and refreshed periodically with new corpus generations.

---

8. Evaluating on your own data

Synthetic benchmarks answer "how does model A compare to model B in a controlled setting?" but not "does this model work well on *our* incidents?" To address the second question, Rhyme includes a web-based labeling tool.

Organizations import their real incidents (as JSON with id, summary, and timestamp — no logs or alerts needed), and the tool generates incident pairs for human evaluation. SREs answer "Would knowing about Incident A help you respond to Incident B?" for each pair, and the tool scores model predictions against human labels with precision, recall, calibration, and red-herring detection.

This two-tool approach — synthetic benchmark for model selection, human labeling for production validation — addresses the gap between controlled evaluation and real-world deployment.

---

9. Conclusion

We presented Rhyme, a benchmark for evaluating LLM-based incident correlation with explicit treatment of the red-herring problem. Testing 9 models from 4 providers revealed three findings with practical implications:

1. **LLMs do not outperform keyword matching on incident retrieval.** All models scored 0.78–0.82 precision@10, clustering around BM25's 0.81. The lexical signal in incident text is strong enough that keyword matching captures most of the retrievable information.

2. **LLMs dramatically outperform keyword matching on remediation.** 72–96% correct vs 16% random, with the ability to distinguish fixes-that-work from fixes-that-worsen. This is the capability that justifies their cost.

3. **Safety-relevant variation exists across models.** DeepSeek V3's 4% harmful-suggestion rate vs 0% for most other models is a meaningful finding that retrieval precision scores alone would miss.

The practical recommendation is a hybrid architecture: use BM25 for candidate retrieval (free, fast, competitive), use LLMs for reasoning-only re-ranking and remediation suggestion (~2,000 tokens per query, 30x cheaper than full-corpus LLM retrieval). This achieves comparable retrieval quality and dramatically better remediation quality at minimal cost.

The benchmark, web labeling tool, and evaluation adapters for all major LLM providers are available as open-source software.

References

- Thakur, N., et al. "BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models." NeurIPS, 2021.
- Muennighoff, N., et al. "MTEB: Massive Text Embedding Benchmark." EACL, 2023.
- Pham, Q. L., et al. "RCAEval: A Benchmark for Root Cause Analysis Methods in Microservice Systems." 2024.
- Microsoft Research. "OpenRCA: Root Cause Analysis Benchmark." ICLR, 2025.
- Yuan, D., et al. "Simple Testing Can Prevent Most Critical Failures." OSDI, 2014.
- Microsoft Research. "What Bugs Cause Production Cloud Incidents?" HotOS, 2019.
- Dan Luu. "Post-mortems." github.com/danluu/post-mortems.
- Atlarge Research. "LLM Cloud Incident Extraction." github.com/atlarge-research/llm-cloud-incident-extraction.